Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.

Workshop #1: Introduction to JADE, Installation, and First Agent

Name: **Cédric Hamel GOUNGOU**

Academic Year 2025-2026

1 Introduction

The objective of this lab is to install the JADE framework, configure the IntelliJ IDEA IDE, and create a first autonomous agent. This assignment helps students understand how multi-agent systems (MAS) work and how they run on a dedicated platform.

2 Adding Libraries to IntelliJ (Equivalent to Build Path)

In IntelliJ IDEA, JAR files are added via:
— File > Project Structure > Libraries
— Adding the jade.jar and commons-codec.jar files

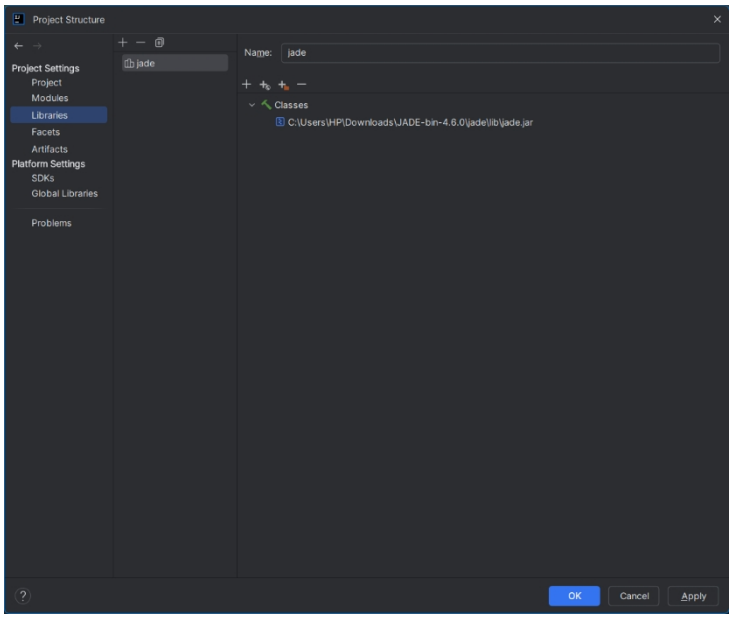

FIGURE 1 – Adding jade.jar and commons-codec.jar to the project libraries (IntelliJ)

3 Development of the HelloAgent Agent

3.1 Source code

```
package   eu. uemf. agents;

import jade.core.Agent;

public class HelloAgent extends Agent {

    @Override
```

```
    protected void setup () {
        System.out.println("Hello World! I am a JADE Agent.");

        // Agent's local name
        System.out.println("My local name is " + getAID().getLocalName());

        // Globally Unique Identifier (GUID)
        System.out.println("My GUID is " + getAID().getName());
    }
}
```

3.2 Technical Explanations

- The HelloAgent class inherits from the Agent class provided by JADE.
- The setup() method represents the agent's entry point (equivalent to main).
- The getAID().getLocalName() and getName() methods identify the agent locally and globally.

4 Run Configuration (IntelliJ)

To run a JADE agent, configure a run with:
- **Main Class**: jade.Boot
- **Program Arguments**:

-gui -local-host 127.0.0.1 -agents myAgent:eu.uemf.agents.HelloAgent

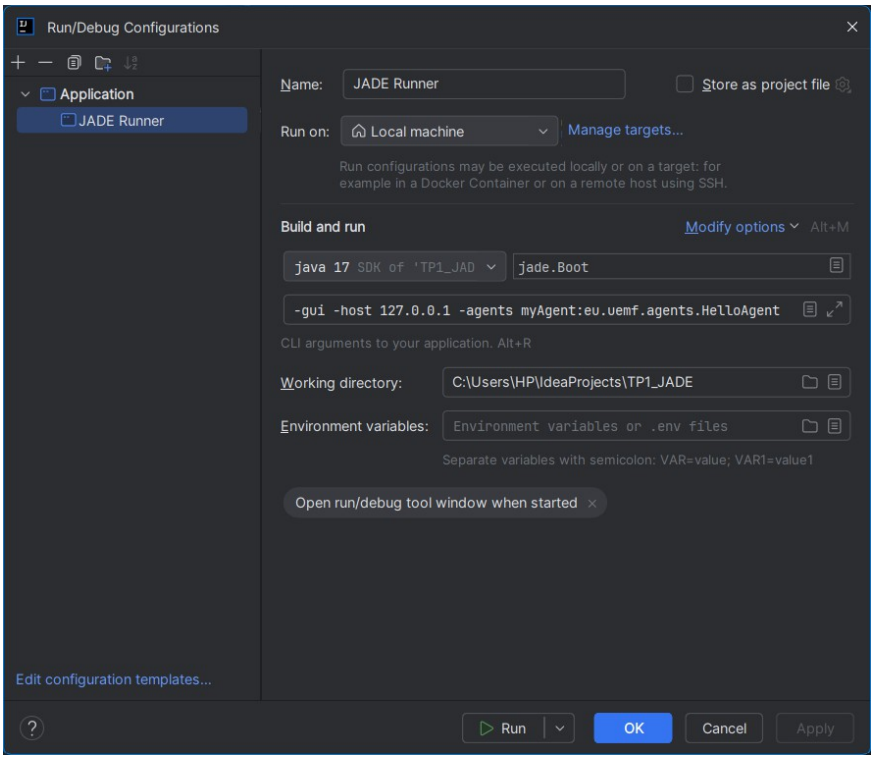

FIGURE 2 – Run configuration in IntelliJ (jade.Boot class and arguments)

5 Execution and Results

5.1 Launching the JADE platform

After configuration, execution launches:

— The Java console displaying agent messages
— The JADE graphical user interface (RMA)

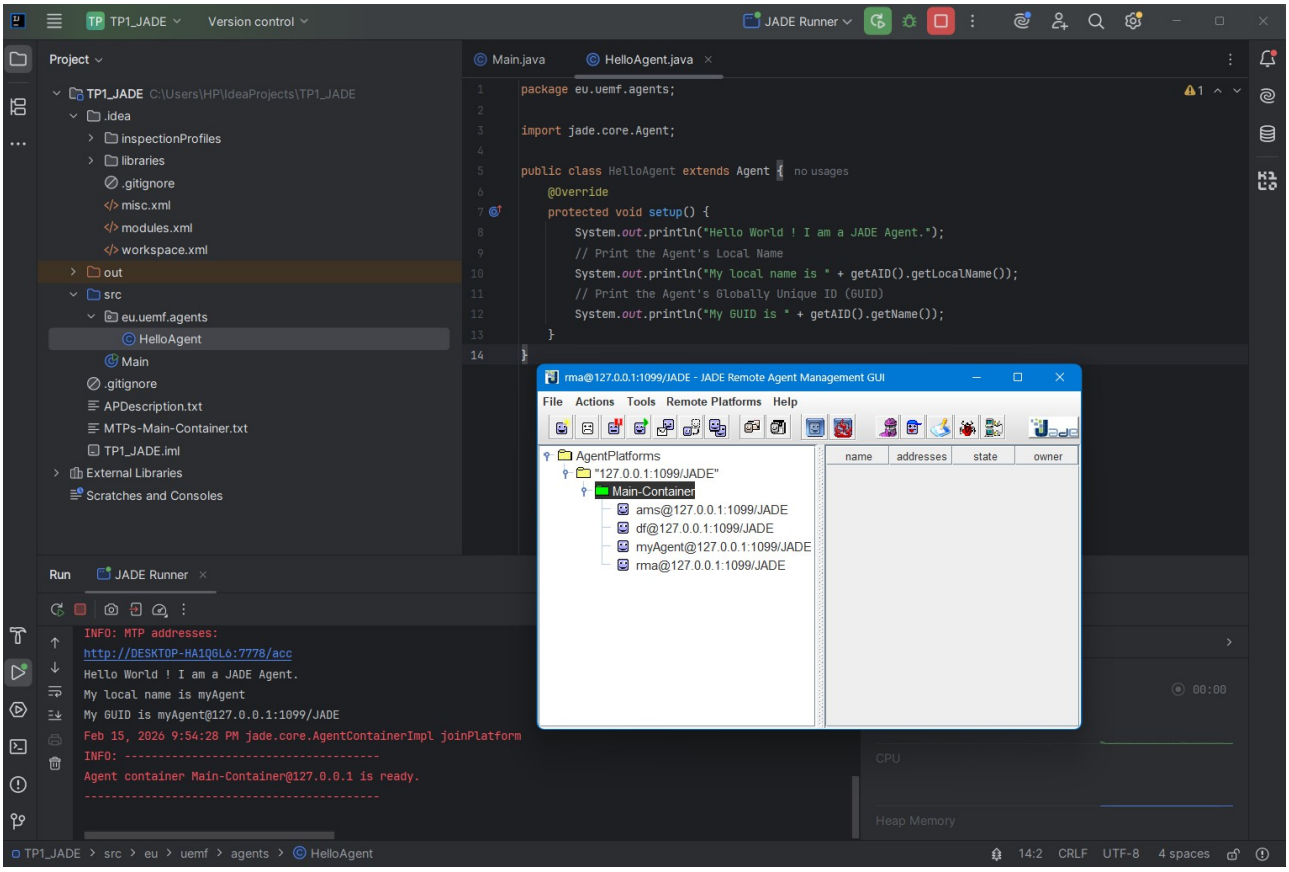

FIGURE 3 – JADE platform execution: console and GUI

5.2 Analysis of the console

The console displays:
— The agent's startup message
— The agent's local name
— Its globally unique identifier (GUID)

5.3 Analysis of the GUI (RMA)

The RMA interface displays:
— **ams**: Agent Management System
— **df**: Directory Facilitator
— **myAgent**: our created agent

6 Part II: Implementation with SPADE (Python)

6.1 Environment setup

IDE used: PyCharm.
Package manager: Using pip in a virtual environment named tp1_mas_venv.
Installation:

```
pip install spade spade_bdi
```

This installation enables support for creating SPADE agents as well as the BDI (Belief-Desire-Intention) reasoning model.

6.2 Communication Protocol (XMPP)

Unlike JADE, SPADE requires the use of an XMPP messaging server to facilitate communication between agents. For this lab, we used the public server:

jabber.hot-chilli.net

Important note: The XMPP protocol (instant messaging) should not be confused with XAMPP, which is a web server used for PHP/MySQL applications.

6.3 Development and Communication

Two agents were implemented:
— **SenderAgent**: sender agent
— **ReceiverAgent**: receiving agent
The code is based on:
— The asyncio library for asynchronous processing
— The Message class for sending and receiving data

6.4 Analysis of the results

During program execution, the following observations were made in the console:
— The receiver agent connects to the XMPP server and waits for messages.
— The XMPP server automatically sends a welcome message.
— The sending agent then sends the message:

"Hi! This is my first SPADE message."

Technical note: The receiver first intercepted the automatic message sent by the XMPP server. The message from the sending agent then triggered the "No behavior matched" alert because the OneShotBehavior had already completed by the time it was received.

7 JADE vs. SPADE Comparison

Criterion	JADE	SPADE
Language	Java	Python
Communication	Internal (Main Container)	External (XMPP Server)
Username	AID (name@host)	JID (user@server, e.g., https://jabber.hot-chilli.net/forms/crea
Parallelism	Java threads	Asynchrony (async/await)

8 General Conclusion

This lab helped us understand the fundamentals of Multi-Agent Systems (MAS). JADE proves to be robust and well-suited for fixed local infrastructures with internal management of communications between agents. In contrast, SPADE offers greater flexibility through the use of Python and the XMPP protocol, which facilitates the future integration of Artificial Intelligence libraries such as torch or scikit-learn, which are already installed in our development environment.